



UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MÉXICO

Centro Universitario UAEM Zumpango

Subida de código de Python usando Git en un
repositorio de Github

Nombre:

Antonio Concepcion Osorio

Profesor:

Hazem Álvarez Rodríguez

Unidad Académica: Sistemas Operativos

Fecha: 15/08/2026

Periodo: 2026B

1. Introducción

El siguiente reporte explica el proceso de configuración de una carpeta local en una computadora para vincularla con un repositorio creado en GitHub. Además, se detalla la subida de un script de Python enfocado en el procesamiento de imágenes para la detección de bordes utilizando el método de Canny. También se documenta el flujo de trabajo para la gestión del código fuente utilizando el sistema de control de versiones Git y su alojamiento en la plataforma GitHub.

2. Entorno de Desarrollo y Configuración

Para ello se utilizó el lenguaje de programación Python junto con las librerías OpenCV (para el procesamiento de imágenes) y Matplotlib (para la visualización).

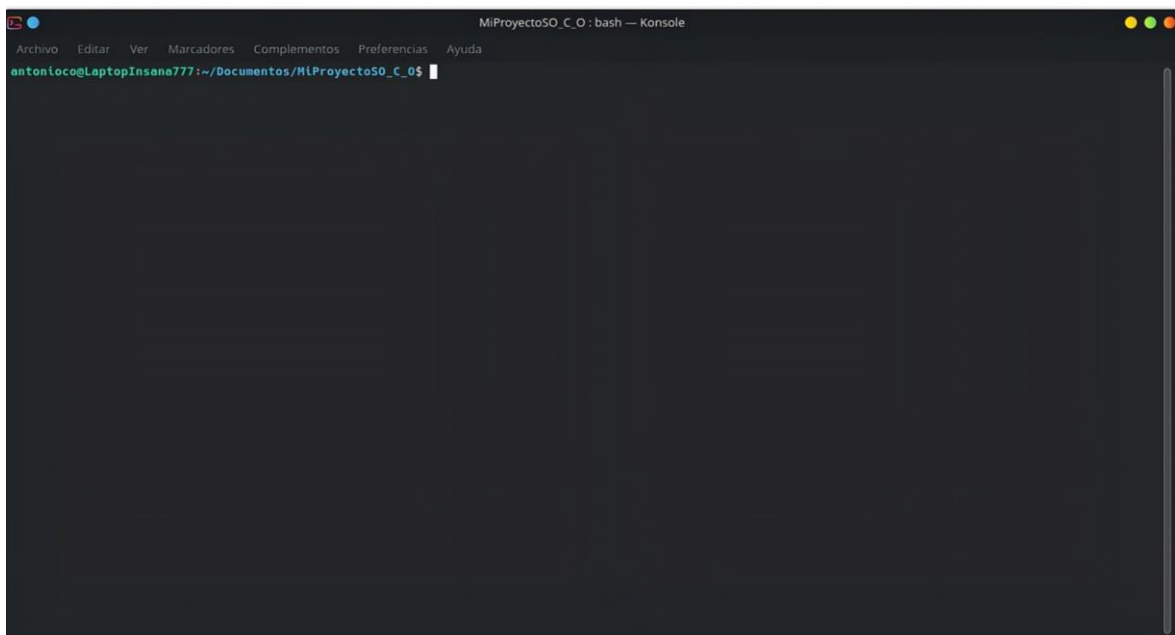
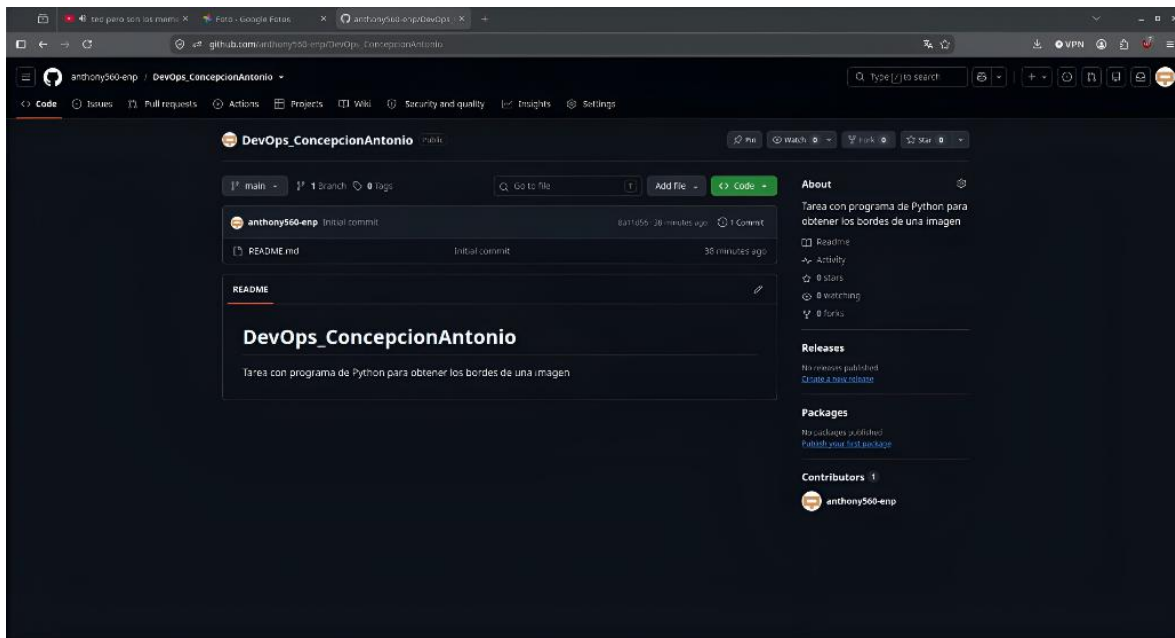
Nota: El entorno de trabajo se configuró en un sistema operativo Linux.

3. Procedimiento

A continuación, se describen los pasos realizados, desde la creación del repositorio en GitHub y la conexión de nuestra carpeta local, hasta la subida del código realizado en Python junto con la imagen resultante.

3.1. Creación del Repositorio y Carpeta Local

Se creó un repositorio público en la plataforma GitHub llamado DevOps_ConcepcionAntonio. De igual manera, en el equipo local se creó una carpeta dedicada para almacenar los archivos del proyecto.

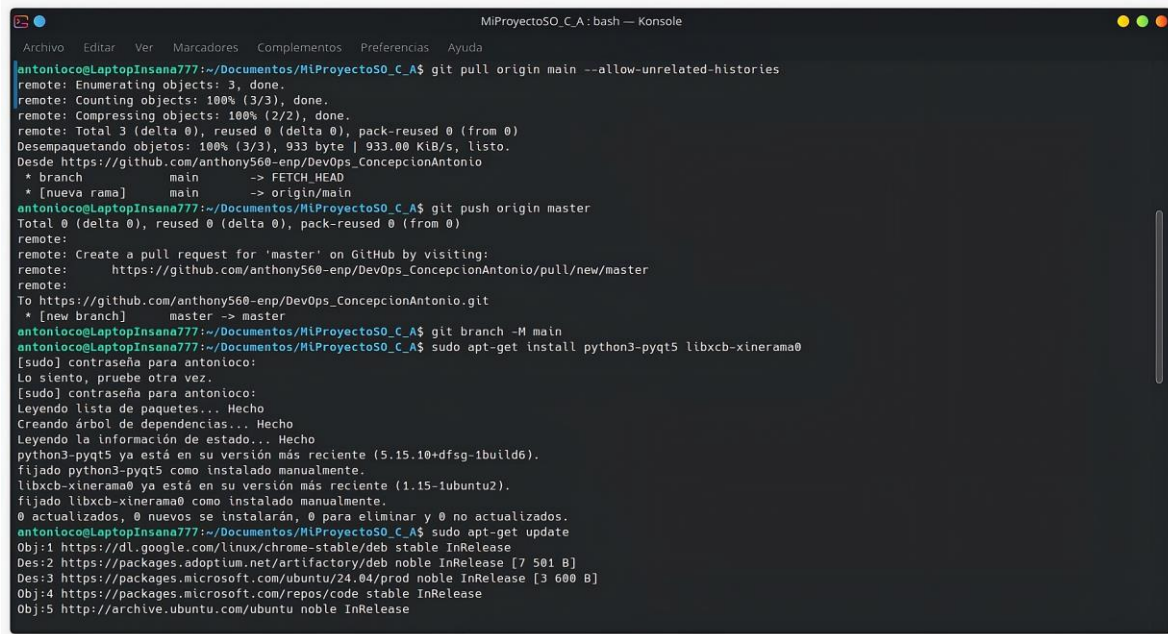


3.2. Inicialización y Sincronización

Contando ya con nuestro repositorio creado en GitHub, nos dirigimos a la terminal de comandos. Se inicializó el control de versiones en la carpeta local con el comando `git init`. Para sincronizar el directorio local con el repositorio remoto, se utilizó el comando `git remote add origin https://github.com/anthony560-emp/DevOps_ConcepcionAntonio`.

Se configuró la rama principal de manera local con el nombre `main` usando el comando `git branch -M main`. Para lograr conectar correctamente ambas partes, se

utilizó el comando `git pull origin main --allow-unrelated-histories`. Este comando sirve para forzar la unión de dos repositorios que no comparten un historial en común; se usa comúnmente para solucionar el error fatal: refusing to merge unrelated histories, el cual no permite la unión entre el repositorio en GitHub y nuestra carpeta local.

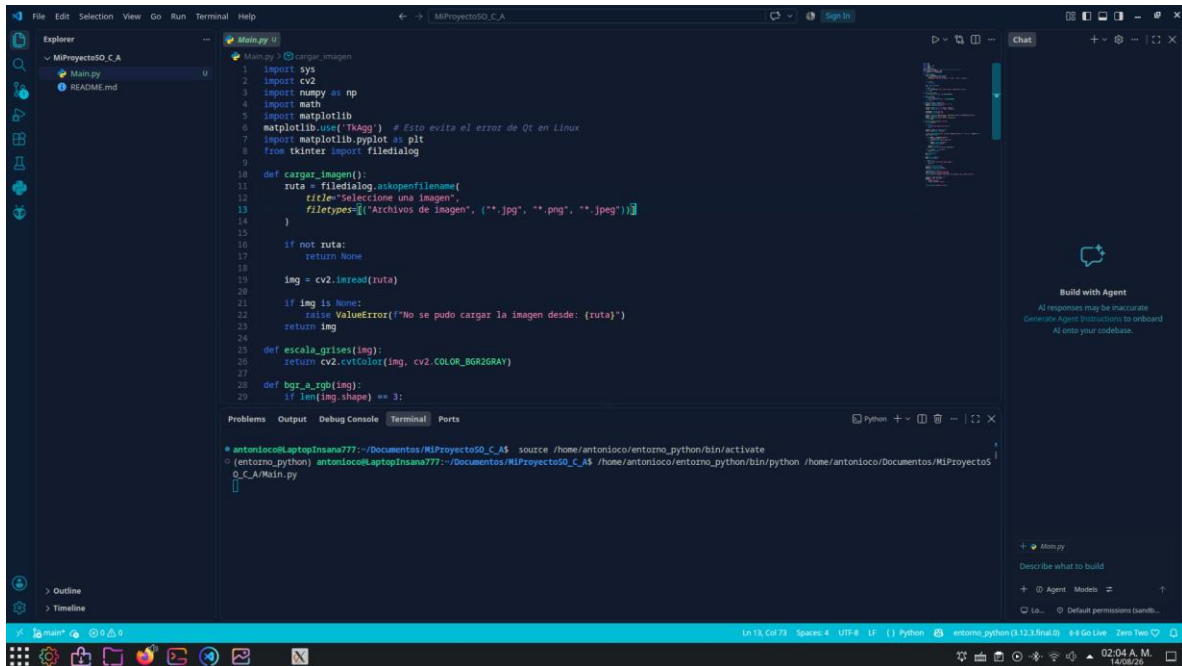


```

MiProyectoSO_C_A: bash — Konsole
Archivo Editar Ver Marcadores Complementos Preferencias Ayuda
antonio@LaptopInsana777:~/Documentos/MiProyectoSO_C_A$ git pull origin main --allow-unrelated-histories
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Desempaquetando objetos: 100% (3/3), 933 byte | 933.00 KiB/s, listo.
Desde https://github.com/anthony560-enp/DevOps_ConcepcionAntonio
* branch      main      -> FETCH_HEAD
* [nueva rama] main      -> origin/main
antonio@LaptopInsana777:~/Documentos/MiProyectoSO_C_A$ git push origin master
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/anthony560-enp/DevOps_ConcepcionAntonio/pull/new/master
remote:
To https://github.com/anthony560-enp/DevOps_ConcepcionAntonio.git
* [new branch]  master -> master
antonio@LaptopInsana777:~/Documentos/MiProyectoSO_C_A$ git branch -M main
antonio@LaptopInsana777:~/Documentos/MiProyectoSO_C_A$ sudo apt-get install python3-pyqt5 libxcb-xinerama0
[sudo] contraseña para antonio:
Lo siento, pruebe otra vez.
[sudo] contraseña para antonio:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
python3-pyqt5 ya está en su versión más reciente (5.15.10+dfsg-1build6).
fijado python3-pyqt5 como instalado manualmente.
libxcb-xinerama0 ya está en su versión más reciente (1.15-1ubuntu2).
fijado libxcb-xinerama0 como instalado manualmente.
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
antonio@LaptopInsana777:~/Documentos/MiProyectoSO_C_A$ sudo apt-get update
Obj:1 https://dl.google.com/linux/chrome-stable/deb stable InRelease
Des:2 https://packages.adoptium.net/artifactory/deb noble InRelease [7 501 B]
Des:3 https://packages.microsoft.com/ubuntu/24.04/prod noble InRelease [3 600 B]
Obj:4 https://packages.microsoft.com/repos/code stable InRelease
Obj:5 http://archive.ubuntu.com/ubuntu noble InRelease
```

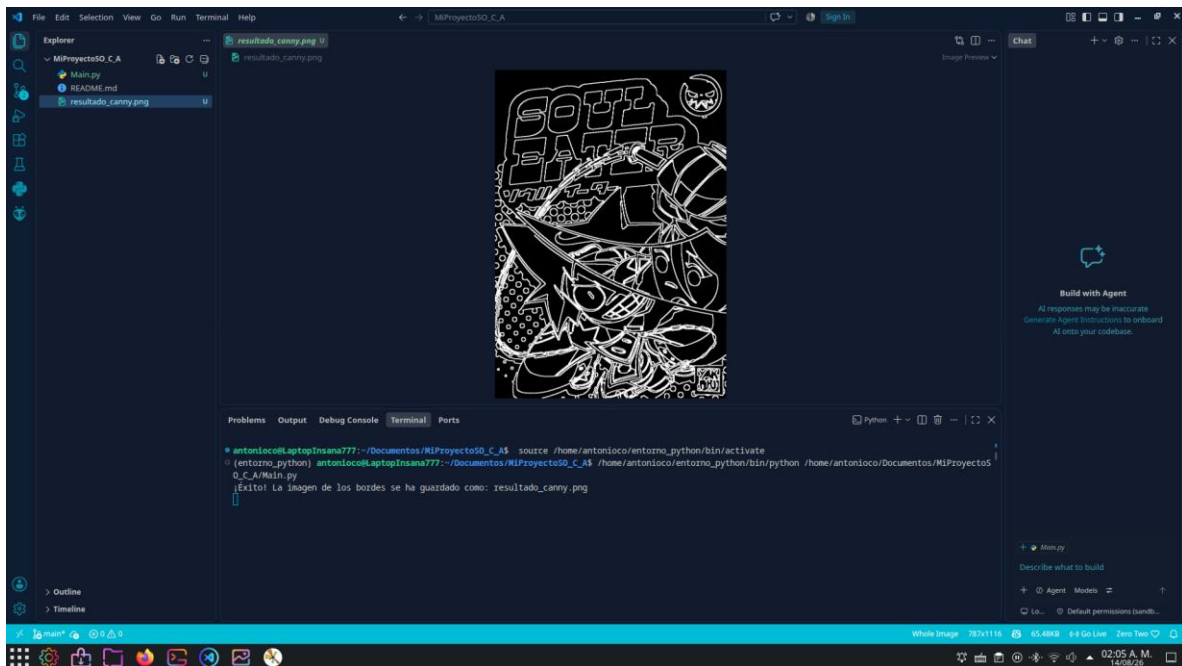
3.3. Desarrollo del Script en Python

Se desarrolló un programa estructurado que permite cargar una imagen desde el sistema de archivos. Una vez cargada, la imagen es convertida a escala de grises y se le aplica un filtro de suavizado Gaussiano. Finalmente, se aplica el método de Canny para detectar los bordes, ajustando los umbrales de manera automática basados en la mediana de la imagen.



3.4. Guardado de Resultados

Como se puede observar en el código anterior, se programó el script para que, al finalizar la ejecución del algoritmo de Canny, el resultado visual se guarde automáticamente en la carpeta del proyecto como un archivo de imagen.



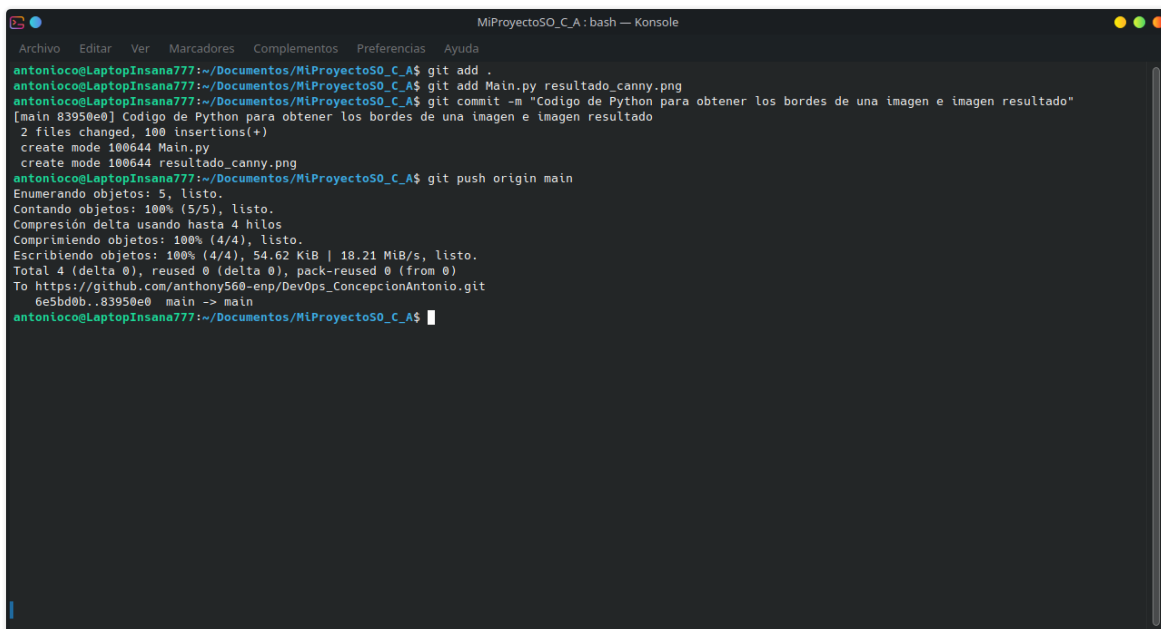
3.5. Subida a GitHub

Una vez generada la imagen del resultado obtenido y verificado el correcto funcionamiento del código, se procedió a subir los cambios al repositorio remoto utilizando los siguientes comandos:

```
git add .
```

```
git commit -m "Código de Python para obtener los bordes de una imagen e imagen resultado"
```

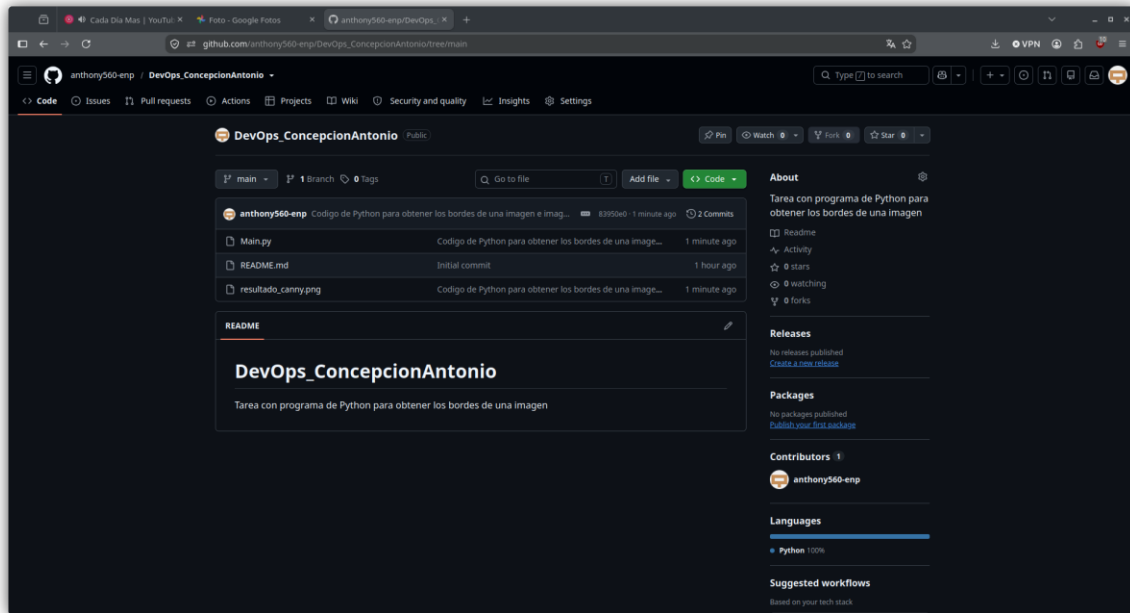
```
git push origin main
```



```

MiProyectoSO_C_A: bash — Konsole
Archivo  Editar  Ver  Marcadores  Complementos  Preferencias  Ayuda
antonio@LaptopInsana777:~/Documentos/MiProyectoSO_C_A$ git add .
antonio@LaptopInsana777:~/Documentos/MiProyectoSO_C_A$ git add Main.py resultado_canny.png
antonio@LaptopInsana777:~/Documentos/MiProyectoSO_C_A$ git commit -m "Codigo de Python para obtener los bordes de una imagen e imagen resultado"
[main 83950e0] Codigo de Python para obtener los bordes de una imagen e imagen resultado
2 files changed, 100 insertions(+)
create mode 100644 Main.py
create mode 100644 resultado_canny.png
antonio@LaptopInsana777:~/Documentos/MiProyectoSO_C_A$ git push origin main
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (4/4), listo.
Escribiendo objetos: 100% (4/4), 54.62 KiB | 18.21 MiB/s, listo.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/anthony560-enp/DevOps_ConcepcionAntonio.git
6e5bd0b..83950e0  main -> main
antonio@LaptopInsana777:~/Documentos/MiProyectoSO_C_A$
```

Al final, para verificar que se subieron nuestros archivos al repositorio de GitHub, basta con simplemente ir a la página, actualizarla y observar que se subió correctamente tanto el código de Python como la imagen resultante.



4. Resultados Obtenidos

La ejecución del script de Python arrojó los resultados esperados. El algoritmo logró procesar la imagen original y resaltar los bordes de los objetos presentes en ella, demostrando el correcto funcionamiento de la librería OpenCV y el método de Canny.



Nota: Se realizó un cambio al código de Python ahora aparte de guardar la imagen resultante también guarda la imagen que se utilizó.

```

MIProyecto50_C_A | bash — Konsole

antonloc@LaptopIsana777:~/Documentos/MIProyecto50_C_A$ cd Documentos/
antonloc@LaptopIsana777:~/Documentos$ ls
Asistencia Dockerfile HW3 MIProyecto50_C_A pr6-vol 'Proyecto Jutsu.pdf' SO-20268
Comandos.txt Estruementa Imagen Ocasion pr6-vol 'Proyecto Aire' Texto.txt
'Comprobante de Apreciación Estudiantil.pdf' Estructura Imagen Pagina.html Practica_23 'Proyecto Ambiental' 'Vision Artificial'
Concepcion-Antonio-SD---20268 Funciones.txt Julia PlatformID Practica4 'Sistemas Embebidos' Web
antonloc@LaptopIsana777:~/Documentos/MIProyecto50_C_A$ ls
imagen_original.png Main.py README.md resultado_canny.png
antonloc@LaptopIsana777:~/Documentos/MIProyecto50_C_A$ git status
En la rama main
Cambios no rastreados para el commit:
  (usa "git add <archivo>..." para actualizar lo que será confirmado)
  (usa "git restore <archivo>..." para descartar los cambios en el directorio de trabajo)
    modificados:   Main.py

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
    imagen_original.png

sin cambios agregados al commit (usa "git add" y/o "git commit -a")
antonloc@LaptopIsana777:~/Documentos/MIProyecto50_C_A$ git add .
antonloc@LaptopIsana777:~/Documentos/MIProyecto50_C_A$ git commit -m "Actualización del código de Python y nuevo archivo imagen original"
[main 52a385c] Actualización del código de Python y nuevo archivo imagen original
 3 files changed, 4 insertions(+),
  create mode 100644 imagen_original.png
antonloc@LaptopIsana777:~/Documentos/MIProyecto50_C_A$ git push
fatal: La rama actual main no tiene una rama upstream.

Para empujar la rama actual y configurar el remoto como upstream, usa

  git push --set-upstream origin main

Para hacer que esto pase automáticamente para las ramas que no rastrean
un upstream, mira 'push.autoSetupRemote' en 'git help config'.

antonloc@LaptopIsana777:~/Documentos/MIProyecto50_C_A$ gh auth
Authenticate gh and git with GitHub

USAGE
  gh auth <command> [flags]

AVAILABLE COMMANDS
  login:      Log in to a GitHub account
  logout:     Log out of a GitHub account
  refresh:    Refresh stored authentication credentials
  setup-gh:   Setup git with GitHub CLI
  status:     View authentication status
  switch:     Switch active GitHub account
  token:      Print the authentication token gh uses for a hostname and account

INHERITED FLAGS
  --help  Show help for command

LEARN MORE
  Use 'gh <command> <subcommand> --help' for more information about a command.
  Read the manual at https://cli.github.com/manual

antonloc@LaptopIsana777:~/Documentos/MIProyecto50_C_A$ gh auth login
? What account do you want to log into? GitHub.com
? What is your preferred protocol for git operations on this host? HTTPS
? How would you like to authenticate GitHub CLI? Login with a web browser

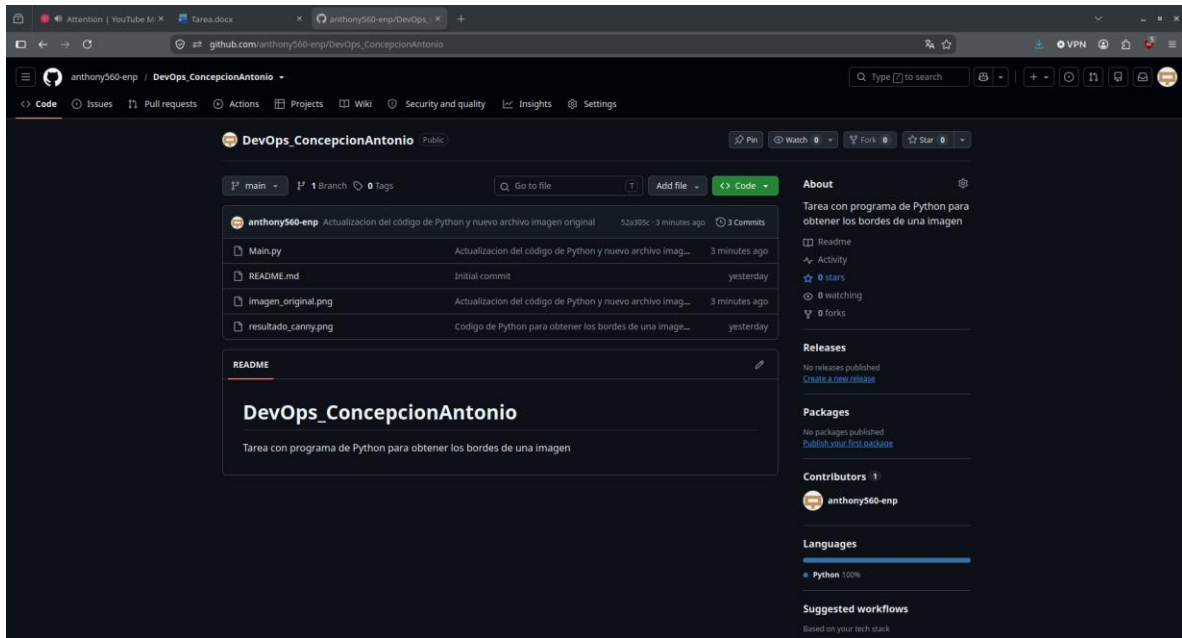
  First copy your one-time code: 7015-3787
  Press Enter to open github.com in your browser...
  Authentication complete.
  - gh config set -h github.com git_protocol https
  - Configured git protocol
  - Logged in as anthony560-emp
  - You were already logged in to this account
antonloc@LaptopIsana777:~/Documentos/MIProyecto50_C_A$ git push
fatal: La rama actual main no tiene una rama upstream.

Para empujar la rama actual y configurar el remoto como upstream, usa

  git push --set-upstream origin main

Para hacer que esto pase automáticamente para las ramas que no rastrean
un upstream, mira 'push.autoSetupRemote' en 'git help config'.

antonloc@LaptopIsana777:~/Documentos/MIProyecto50_C_A$ git push --set-upstream origin main
Enumerando objetos: 6, listo.
Contando objetos: 100% (6/6), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (4/4), listo.
Escribiendo objetos: 100% (4/4), 1.05 MiB | 17.63 MiB/s, listo.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com:anthony560-emp/DevOps_ConcepcionAntonio.git
  83958ee..52a385c  main -> main
rama 'main' configurada para rastrear 'origin/main'.
antonloc@LaptopIsana777:~/Documentos/MIProyecto50_C_A$
```



5. Conclusiones

El uso de Git y GitHub permitió mantener un control ordenado de las versiones del código, asegurando que tanto el script de Python como las evidencias gráficas del resultado quedaran almacenadas de manera segura y accesible en la nube. Además, se logró comprender y aplicar el flujo de trabajo básico para la gestión de proyectos de desarrollo de software.